SOUTHEAST UNIVERSITY

Meeting the Challenges of Time

Department Of CSE

Assignment

Assignment No:02

Date of submission: 1 June 2026

Course name : CSE LAB

Course code : CSE362

Section : 6

Submitted To : Shujoy Kundu

[lecturer, Department of CSE]

Initial : **SHKU**

Student's Name : Nazmul Hasan

Student's ID : 2023100000130

Shell Programming, GREP & Basic Networking

Program 1 - Demonstrates the use of comments, user-defined variables and echo.

Explain:

- echo is used for printing any text in terminal.
- $ this is used to call the userdefine variable.
- # is used for comment in bash script.
- #!/bin/bash tells the system to run the script.

Code:

```
1. #!/bin/sh
2. #Filename: shelldemo1  Author: RS
3. #Define variables
4. name=John
5. car="Ford Escort"
6. age=21
7. #Display the contents of the variables
8. echo "My name is $name"
9. echo "I am $age years old and \c"
10.      echo "I drive a $car."
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo1.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo1.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo1.sh
./demo1.sh: line 1: i#!/bin/sh: No such file or directory
My name is Nazmul
I am 21 years old and I drive a Ford Escort.
```

Program 2 - Demonstrates how the output of Unix commands can be stored in user-defined variables

Explain:

- `` (backticks) are used for command substitution, meaning it runs the command inside and captures its output.
- `date` is a built-in command that fetches the current system date and time.
- `pwd` is a command that prints the current working directory path.

- todaysdate stores the output generated by the command directly into the variable.

Code:

```
1. #!/bin/sh
2. #Filename: shelldemo2  Author: RS
3. #Define variables
4. todaysdate=`date`
5. myworkingdirectory=`pwd`
6. #Display the contents of the variables
7. echo "The date is $todaysdate"
8. echo "My current working directory is $myworkingdirectory"
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo2.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo2.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo2.sh
The date is Sun May 31 10:32:24 +06 2026
My current working directory is /home/nazmul/desktop/cse362.6/documents/assignment1
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Program 3 - Demonstrating how the read command is used to get input from the user via the keyboard.

Explain:

- read is used to take input from the user via the keyboard and store it in a variable.
- \c is used with echo to keep the cursor on the same line (prevents moving to a new line) so the user types right next to the prompt.
- firstname and surname are user-defined variables created to store the user's keyboard input.

Code:

```
1. #!/bin/sh
2. #Filename: shelldemo3  Author:  RS
3. echo "Please enter your first name:  \c"
4. read firstname
5. echo "Please enter your surname:  \c"
6. read surname
7. echo "Please enter your date of birth: (dd/mm/yyyy):  \c"
8. read dateofbirth
9. echo "Welcome $firstname $surname, your date of birth is on record
   as $dateofbirth."
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo3.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo3.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo3.sh
-e Please enter your first name: Nazmul
-e Please enter your surname: Hasan
-e Please enter your date of birth (dd/mm/yyyy): 19/12/2002
Welcome Nazmul Hasan, your date of birth is on record as 19/12/2002.
```

Program 4 - Demonstrating how user input can be obtained from the command line as command line arguments.

Explain:

- $1, $2, $3 are positional variables representing the 1st, 2nd, and 3rd words passed to the script from the command line.

Code:

```
1. #!/bin/sh
2. #Filename: shelldemo4  Author:  RS
3. echo "This program has obtained its input from the command line"
4. echo "Welcome $1 $2, your date of birth is on record as $3."
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo4.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo4.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo4.sh Nazmul Hasan 19/12/2002
This program has obtained its input from the command line
Welcome Nazmul Hasan, your date of birth is on record as 19/12/2002.
```

Program 5 - Demonstrating how decisions are madeusing the if...then statement (testing and branching).

Explain:

- if [condition] is used to check if a specific condition is true .
- "$reply" = "y" checks if the value typed by the user exactly matches the letter "y".
- then specifies the block of code to execute if the condition is true.
- fi is used to close/end the if statement block.

Code:

```
1. #!/bin/sh
2. #Filename: shelldemo5  Author:  RS
3. clear
4. echo "Would you like to see a joke (y/n)? \c"
5. read reply
6. if [ "$reply" = "y" ]
7. then
8.     echo "Question: How many surrealists does it take to change a light
   bulb?"
9.     echo "Answer: Fish."
10.       fi
11.       echo "\n\nHave a nice day."
```

Output:

```
-e Would you like to see a joke (y/n)? y
Question: How many surrealists does it take to change a light bulb?
Answer: Fish.
-e

Have a nice day.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Program 6 - Demonstrating how decisions are made using the if...then...else statement (testing and branching).

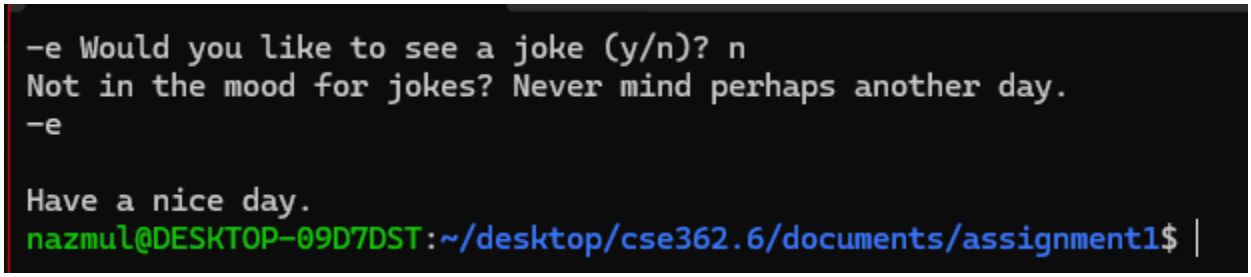
Explain:

- if [condition] is used to check if a specific condition is true .
- else provides an alternative block
- fi is used to close/end the if statement block.

Code:

```
1. #!/bin/sh
2. #Filename: shelldemo6  Author:  RS
3. clear
4. echo "Would you like to see a joke (y/n)? \c"
5. read reply
6. if [ "$reply"  =  "y" ]
7. then
8.     echo "Question: How many surrealists does it take to change a light
   bulb?"
9.     echo "Answer: Fish."
10.       else
11.echo "Not  in  the  mood  for  jokes?  Never  mind  perhaps
   another day."
12.       fi
13.       echo "\n\nHave a nice day."
```

Output:



Program 7 - Demonstrating how decisions are made using nested if...then...else statements (testing and branching).

Explain:

- elif stands for "else if" and allows the program to check multiple conditions one after another.
- -eq is a numerical comparison operator used to check if two numbers are exactly equal.
- \07\07 represents the ASCII bell character, which makes the computer beep when an invalid choice is selected.

Code:

```
1. #!/bin/sh
2. #Filename: shelldemo7  Author:  RS

3. echo "UNIX COMMAND SELECTOR"
4. echo "1. Show date"
5. echo "2. Show hostname"
6. echo "3. Show this month's calendar"
7. echo "Please make your selection (1,2,3) \c"
8. read menunumber
9. if  [ $menunumber -eq 1 ]
10.       then
11.           date
12.       else if  [ $menunumber -eq 2 ]
13.               then
14.                   hostname
15.               else if [ $menunumber -eq 3 ]
16.                       then
17.                           cal
18.                       else
19.                           echo "INVALID CHOICE! \07\07"
20.                       fi
21.                fi
22.       fi
23.       echo "\nThank you for using the Unix command selector."
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo7.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo7.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo7.sh
UNIX COMMAND SELECTOR
1. Show date
2. Show hostname
3. Show this month's calendar
-e Please make your selection (1,2,3) 1
Sun May 31 13:49:43 +06 2026
-e
Thank you for using the Unix command selector.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo7.sh
UNIX COMMAND SELECTOR
1. Show date
2. Show hostname
3. Show this month's calendar
-e Please make your selection (1,2,3) 2
DESKTOP-09D7DST
-e
Thank you for using the Unix command selector.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo7.sh
UNIX COMMAND SELECTOR
1. Show date
2. Show hostname
3. Show this month's calendar
-e Please make your selection (1,2,3) 3
      May 2026
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
-e
Thank you for using the Unix command selector.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Program 8 - Demonstrating how decisions are made using the case...esac statement (testing and branching).

Explain:

- case ... in starts a multiple-choice selection, which is a cleaner alternative to writing many if-else blocks.

- ;; (double semicolons) are used to indicate the end of a specific case block.
- *) acts as a default catch-all (like else) if none of the above options match.
- esac closes the statement

Code:

```
1. echo "UNIX COMMAND SELECTOR"
2. echo "1. Show date"
3. echo "2. Show hostname"
4. echo "3. Show this month's calendar"
5. echo "Please make your selection (1,2,3) \c"
6. read menunumber
7. case $menunumber in
8.       1)  date;;
9.       2)  hostname;;
10.           3)  cal;;
11.           *)  echo "INVALID CHOICE! \07\07";;
12.      esac
13.      echo "\nThank you for using the Unix command selector."
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo8.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo8.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo8.sh
UNIX COMMAND SELECTOR
1. Show date
2. Show hostname
3. Show this month's calendar
-e Please make your selection (1,2,3) 1
Sun May 31 13:53:05 +06 2026
-e
Thank you for using the Unix command selector.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo8.sh
UNIX COMMAND SELECTOR
1. Show date
2. Show hostname
3. Show this month's calendar
-e Please make your selection (1,2,3) 2
DESKTOP-09D7DST
-e
Thank you for using the Unix command selector.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo8.sh
UNIX COMMAND SELECTOR
1. Show date
2. Show hostname
3. Show this month's calendar
-e Please make your selection (1,2,3) 3
      May 2026
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
-e
Thank you for using the Unix command selector.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Program 9 - Demonstrating how looping is achieved using the for statement.

Explain:

- `for variable in list` starts a loop that will repeat for every item provided in the list.
- `do` indicates the start of the commands that will be executed in each loop cycle.
- `done` marks the end of the loop block.
- In this program, the list is a hardcoded set of car names separated by spaces.

Code:

```
1. echo "Demonstration of looping using the for loop and a list of car
   names"
2. for car in ford vauxhall rover toyota mazda subaru
3. do
4.     echo $car
5. done
6. echo "\nEnd of demonstration program."
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo9.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo9.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo9.sh
Demonstration of looping using the for loop and a list of car names
ford
vauxhall
rover
toyota
mazda
subaru
-e
End of demonstration program.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Program 10 - Demonstrating how looping is achieved using the for statement.

Explain:

- `` ` `` (backticks) are used to execute the ls command and generate a list of files in the current directory.
- The for loop dynamically iterates through each filename generated by the command.
- `cat $myfile` is used inside the loop to display the text content of each file found.

Code:

```
1. echo "Demonstration of looping using the for loop and a list of
   filenames generated by the ls command"
2. for myfile in `ls`
3. do
4.     cat $myfile
5. done
```

Output:

```
ls: cannot access '*.txt': No such file or directory
nazmul@DESKTOP-69070ST:~/desktop/cse362.6/documents/assignment1$ vim demo10.sh
nazmul@DESKTOP-69070ST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo10.sh
nazmul@DESKTOP-69070ST:~/desktop/cse362.6/documents/assignment1$ ./demo10.sh
Demonstration of looping using the for loop and a list of filenames generated by the ls command
i#!/bin/sh
#Filename: shelldemo1 Author: RS
name=Nazmul
car='Ford Escort'
age=21
echo 'My name is $name'
echo -e 'I an $age years old and \c'
echo 'I drive a $car.'

#!/bin/sh
echo "Demonstration of looping using the for loop and a list of filenames generated by the ls command"
for myfile in `ls`
do
    cat $myfile
done
#!/bin/sh
#Filename: shelldemo2 Author: RS
todaysdate=`date`
myworkingdirectory=`pwd`
echo 'The date is $todaysdate'
echo 'My current working directory is $myworkingdirectory'
#!/bin/sh
#Filename: shelldemo3 Author: RS
echo -e 'Please enter your first name: \c'
read firstname
echo -e 'Please enter your surname: \c'
read surname
echo -e 'Please enter your date of birth (dd/mm/yyyy): \c'
read dateofbirth
echo 'Welcome $firstname $surname, your date of birth is on record as $dateofbirth.'
#!/bin/sh
#Filename: shelldemo4 Author: RS
echo 'This program has obtained its input from the command line'
echo 'Welcome $1 $2, your date of birth is on record as $3.'
#!/bin/sh
#Filename: shelldemo5 Author: RS
clear
echo -e 'Would you like to see a joke (y/n)? \c'
read reply
if [ "$reply" = "y" ]
then
    echo 'Question: How many surrealists does it take to change a light bulb?'
    echo 'Answer: Fish.'
fi
echo -e '\n\nHave a nice day.'
#!/bin/sh
#Filename: shelldemo6 Author: RS
clear
echo -e 'Would you like to see a joke (y/n)? \c'
read reply
if [ "$reply" = "y" ]
then
    echo 'Question: How many surrealists does it take to change a light bulb?'
    echo 'Answer: Fish.'
else
    echo 'Not in the mood for jokes? Never mind perhaps another day.'
fi
echo -e '\n\nHave a nice day.'
#!/bin/sh
#Filename: shelldemo7 Author: RS
echo 'UNIX COMMAND SELECTOR'
echo '1. Show date'
echo '2. Show hostname'
echo '3. Show this month's calendar'
echo -e 'Please make your selection (1,2,3) \c'
read menunumber

if [ $menunumber -eq 1 ]
then
    date
elif [ $menunumber -eq 2 ]
then
    hostname
elif [ $menunumber -eq 3 ]
then
    cal
else
    echo -e 'INVALID CHOICE! \07\07'
fi
echo -e '\nThank you for using the Unix command selecter.'
#!/bin/sh
echo 'UNIX COMMAND SELECTOR'
echo '1. Show date'
echo '2. Show hostname'
echo '3. Show this month's calendar'
echo -e 'Please make your selection (1,2,3) \c'
read menunumber
case $menunumber in
    1) date;;
    2) hostname;;
    3) cal;;
    *) echo -e 'INVALID CHOICE! \07\07';;
esac
echo -e '\nThank you for using the Unix command selecter.'
#!/bin/sh
echo "Demonstration of looping using the for loop and a list of car names"
for car in ford vauxhall rover toyota mazda subaru
do
    echo $car
done
echo -e '\nEnd of demonstration program.'
nazmul@DESKTOP-69070ST:~/desktop/cse362.6/documents/assignment1$
```

Program 11 - Demonstrating how looping is achieved using the for statement.

Explain:

- `cat myfilelist` reads the contents of an external text file to generate the list of items for the loop.
- This demonstrates how to feed a loop with a list stored in a file rather than typing the list directly into the script.

Code:

```
1. echo "Demonstration of looping using the for loop and a list of
   filenames held in a text file named myfilelist"
2. for myfile in `cat myfilelist `
3. do
4.     cat $myfile
5. done
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo11.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo11.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo11.sh
Demonstration of looping using the for loop and a list of filenames held in a text file named myfilelist
cat: myfilelist: No such file or directory
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Program 12 - Demonstrating how looping is achieved using the for statement.

Explain:

- $* is a special variable that represents *all* the arguments provided by the user on the command line as a single list.
- The for loop takes this $* list and iterates through each passed argument, allowing the user to provide an unknown amount of filenames when running the script.

Code:

```
1. echo "Demonstration of looping using the for loop and a list of
   arguments supplied at the command line"
2. echo "Program invoked by the command:   ./shelldemo12  filename1
   filename2  filename3"
3. for myfile in $*
4. do
5.     cat $myfile
6. done
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo12.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo12.sh demo1.sh demo2.sh demo3.sh
Demonstration of looping using the for loop and a list of arguments supplied at the command line
Program invoked by the command: ./demo12.sh demo1.sh demo2.sh demo3.sh
--- Contents of demo1.sh ---
i#!/bin/sh
#Filename: shelldemo1 Author: RS
name=Nazmul
car="Ford Escort"
age=21
echo "My name is $name"
echo -e "I am $age years old and \c"
echo "I drive a $car."

--- Contents of demo2.sh ---
#!/bin/sh
#Filename: shelldemo2 Author: RS
todaysdate=`date`
myworkingdirectory=`pwd`
echo "The date is $todaysdate"
echo "My current working directory is $myworkingdirectory"
--- Contents of demo3.sh ---
#!/bin/sh
#Filename: shelldemo3 Author: RS
echo -e "Please enter your first name: \c"
read firstname
echo -e "Please enter your surname: \c"
read surname
echo -e "Please enter your date of birth (dd/mm/yyyy): \c"
read dateofbirth
echo "Welcome $firstname $surname, your date of birth is on record as $dateofbirth."
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Program 13 - Demonstration of 'while' loop

Explain:

Code:

```
#!/bin/bash
n=1
while (( $n <= 5 ))
do
        echo "Welcome $n times."
        n=$(( n+1 ))
done
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ vim demo13.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ chmod +x demo13.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$ ./demo13.sh
Welcome 1 times.
Welcome 2 times.
Welcome 3 times.
Welcome 4 times.
Welcome 5 times.
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment1$
```

Sample bash program

Problem 1: Count number of files and directories in current folder#! /bin/bash

Explain:

- path=$(pwd) captures the current working directory path and stores it in the path variable.
- for i in $content iterates through every item found by the ls command.
- count=$(($count+1)) is arithmetic expansion used to increase the total item counter by 1.
- -f "$i" is a conditional flag that checks if the current item is a **regular file**.
- -d "$i" is a conditional flag that checks if the current item is a **directory** (folder).
- ((files++)) and ((directories++)) are shorthand ways to increment their respective counters.

Code:

```
#! /bin/bash
# filename: countnumberoffiles  author:AlfaSnitch
#
path=$(pwd)
content=$(ls $path)
count=0
files=0
directories=0
for i in $content
do
        count=$(($count+1))
        if [ -f "$i" ]; then
                ((files++))
        elif [ -d "$i" ]
        then
                ((directories++))
        fi
done
echo "$count of files and directories"
echo "number of files $files"
echo "number of directories $directories"
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ vim problem1.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ chmod +x problem1.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ./problem1.sh
1 of files and directories
number of files 1
number of directories 0
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$
```

Problem 2: Create a script that backs up all the documents

Explain:

- `mkdir -p backup` creates a new folder named "backup". The `-p` flag prevents errors if the folder already exists.
- `for file in *` iterates over every single item (*) in the current directory.
- `||` is the logical OR operator. The if statement checks if the current file is the script itself ($0) OR the "backup" folder.
- `continue` tells the loop to skip the current item and move to the next one (prevents the script from backing up itself or creating an infinite loop).
- `cp "$file" backup/` copies regular files to the backup folder.
- `cp -r "$file" backup/` uses `-r` (recursive) to copy entire directories and their contents

Code:

```
#! /bin/bash
#
#filename:backup.sh            author:AlfaSnitch
#
mkdir -p backup
for file in *
do
        if [ "$file" = "$0" ] || [ "backup" = "$file" ]; then
                continue
        elif [ -f "$file" ]; then
                cp "$file" backup/
        elif [ -d "$file" ]; then
                cp -r "$file" backup/
        elif [ -e "$file" ]; then
                cp "$file" backup/
        else
                echo "$file not supported"
        fi
done
echo "backup is complete"
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ vim problem2.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ chmod +x problem2.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ./problem2.sh
backup is complete
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ls
backup  problem1.sh  problem2.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ls backup
problem1.sh  problem2.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$
```

Problem 3: Check Systeminfo

Explain:

- $USER is a built-in environment variable that holds the current logged-in username.
- $(hostname) runs the command to fetch the computer's network name.
- $(uptime -p) prints how long the system has been running in a pretty, human-readable format.
- $(ls | wc -l) combines two commands: ls lists items, and wc -l counts the number of lines, effectively counting the files/folders in the directory.
- df -h displays the disk space usage in human-readable sizes (like MB or GB).

Code:

```
#!/bin/bash
# filename: systeminfo.sh                    #author: AlfaSnitch
echo "User: $USER"
echo "Hostname: $(hostname)"
echo "Uptime: $(uptime -p)"
echo "Current Directory: $(pwd)"
echo "Files in Directory: $(ls | wc -l)"
echo "Disk Usage:"
df -h
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ vim problem3.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ chmod +x problem3.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ./problem3.sh
User: nazmul
Hostname: DESKTOP-09D7DST
Uptime: up 3 hours, 1 minute
Current Directory: /home/nazmul/desktop/cse362.6/documents/assignment2
Files in Directory: 4
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
none            4.7G     0  4.7G   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            4.7G  4.0K  4.7G   1% /mnt/wsl
drivers         162G  143G   19G  89% /usr/lib/wsl/drivers
/dev/sdd       1007G  7.6G  949G   1% /
none            4.7G  404K  4.7G   1% /mnt/wslg
none            4.7G     0  4.7G   0% /usr/lib/wsl/lib
rootfs          4.7G  2.7M  4.7G   1% /init
none            4.7G  960K  4.7G   1% /run
none            4.7G     0  4.7G   0% /run/lock
none            4.7G     0  4.7G   0% /run/shm
none            4.7G  1.6M  4.7G   1% /mnt/wslg/versions.txt
none            4.7G  1.6M  4.7G   1% /mnt/wslg/doc
C:\             162G  143G   19G  89% /mnt/c
D:\             165G   42G  123G  26% /mnt/d
E:\             151G   90G   61G  60% /mnt/e
snapfuse        128K  128K     0 100% /snap/bare/5
snapfuse         74M   74M     0 100% /snap/core22/2292
snapfuse         74M   74M     0 100% /snap/core22/2411
snapfuse         67M   67M     0 100% /snap/core24/1587
snapfuse         67M   67M     0 100% /snap/core24/1643
snapfuse        252M  252M     0 100% /snap/firefox/7836
snapfuse        249M  249M     0 100% /snap/firefox/8387
snapfuse        395M  395M     0 100% /snap/mesa-2404/1165
snapfuse        607M  607M     0 100% /snap/gnome-46-2404/153
snapfuse        532M  532M     0 100% /snap/gnome-42-2204/247
snapfuse         92M   92M     0 100% /snap/gtk-common-themes/1535
snapfuse         49M   49M     0 100% /snap/snapd/25935
snapfuse         50M   50M     0 100% /snap/snapd/26865
snapfuse        228M  228M     0 100% /snap/thunderbird/1001
snapfuse        228M  228M     0 100% /snap/thunderbird/1057
tmpfs           961M  128K  961M   1% /run/user/1000
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$
```

Problem 4: Ping a host and check connectivity

Explain:

- `read -p` prints the prompt text and waits for the user's input on the exact same line.
- `-c 1` tells the ping command to only send **one** network packet instead of pinging infinitely.
- `> /dev/null` acts as a "black hole" that hides all the messy output of the ping command so the terminal stays clean.
- `$?` holds the exit status of the very last command run. `0` means success (the host replied), and non-zero means failure

Code:

```
#!/bin/bash
read -p "Enter host (e.g. google.com): " host
ping -c 1 "$host" > /dev/null
if [ $? -eq 0 ]; then
  echo "$host is reachable"
else
  echo "$host is not reachable"
fi
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ vim problem4.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ chmod +x problem4.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ./problem4.sh
Enter host (e.g. google.com): seu.edu.bd
seu.edu.bd is reachable
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$
```

Problem 4: Port Scanning

Explain:

- {20..25} generates a sequence of numbers from 20 to 25 for the loop to iterate through.
- /dev/tcp/$host/$port uses a special Bash feature to attempt a raw TCP connection (a "knock") to a specific port on the target machine.
- `timeout 1` forces the command to stop if it takes longer than 1 second, preventing the script from hanging on unresponsive ports.
- `2>/dev/null` hides any error messages (like "Connection Refused") so the user only sees clean output.

Code:

```
#!/bin/bash
read -p "Enter host: " host
for port in {20..25}; do
  timeout 1 bash -c "echo > /dev/tcp/$host/$port" 2>/dev/null
  if [ $? -eq 0 ]; then
    echo "Port $port is OPEN"
  else
    echo "Port $port is CLOSED"
  fi
done
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ vim problem4_1.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ chmod +x problem4_1.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ./problem4_1.sh
Enter host: localhost
Port 20 is CLOSED
Port 21 is CLOSED
Port 22 is CLOSED
Port 23 is CLOSED
Port 24 is CLOSED
Port 25 is CLOSED
```

Problem 5: Monitor Internet Connection Continuously

Explain:

- `while true; do` creates an infinite loop that will run forever until the user manually stops it.
- `8.8.8.8` is Google's highly reliable public DNS server, used here as a landmark to test general internet access.
- `sleep 5` pauses the script for 5 seconds between each loop to prevent spamming the network with ping requests.

Code:

```
#!/bin/bash
while true; do
  ping -c 1 8.8.8.8 > /dev/null
  if [ $? -eq 0 ]; then
    echo "Internet is UP"
  else
    echo "Internet is DOWN"
  fi
  sleep 5
done
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ vim problem5.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ chmod +x problem5.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ./problem5.sh
Internet is UP
Internet is UP
Internet is UP
^C
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$
```

Problem 6: Network Surveillance

Explain:

- `netstat` is a network utility used to view active connections and routing statistics.
- `-tun` are flags that tell netstat to show **T**CP connections, **U**DP connections, and display IP addresses as **N**umbers

Code:

```
#!/bin/bash
echo "Active connections:"
netstat -tun | grep ESTABLISHED
```

Output:

```
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ vim problem6.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ chmod +x problem6.sh
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$ ./problem6.sh
Active connections:
nazmul@DESKTOP-09D7DST:~/desktop/cse362.6/documents/assignment2$
```